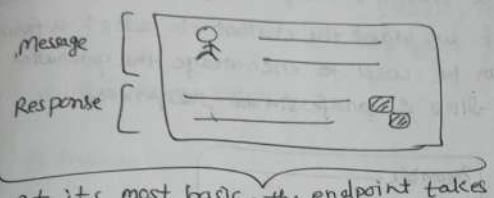
At its most basic, we only need to pass to the chat endpoint the user message using the <u>message</u> parameter — the only required parameter for the endpoint.



At its most basic, the endpoint takes a user message & generates a response

↳ Here's an example. We call the endpoint with "Hello" as the user message. In our case, we also use the <u>model</u> parameter to specify the underlying chat model we want to use. For this, we pick Command R+, Cohere's newest LLM.

Right now, we're interested in the main content of the response, which is stored in the <u>text</u> value of the response.

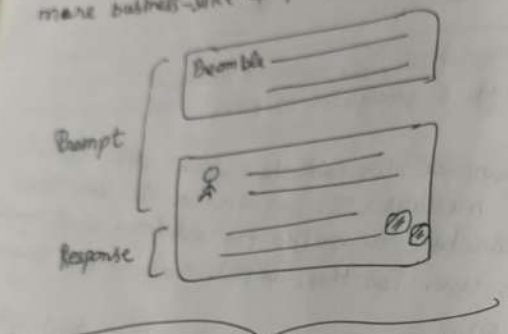
```
response = co.chat(message="Hello", model="command-r-plus")
print(response.text)
```

<u>O/P</u>

```
#Response
Hello! How can I help you today?
```

↳ Step ②: Defining a Preamble

A conversation starts with a system message, or a preamble, to help steer a chatbot's response toward certain characteristics. For example, if we want the chatbot to adopt a formal style, the preamble can be used to encourage the generation of more business-like & professional responses.



The preamble & the current user message form the prompt to the chatbot

In the quickstart example, we didn't have to define a preamble because a default one was used. We can, however, define our own preamble using the "preamble" parameter.

Here's an example. We added a preamble telling the chatbot to assume the persona of an expert public speaking coach. As a result, we get a response that adopts that persona.

response = co.

print (respon

O/P # Response

How can I

↳ Step ③: Streami

Our examples so

manner. This means

object only after

The longer the text

response. If you

the user's percep

↳ The Chat endpoin

stream responses

would return

is being genera

incrementally

completion.

```
response = co.chat(message= "Hello",
                   model= "command-r-plus",
                   preamble= "You are an expert public
                              speaking coach. Don't use
                              any greetings.")

print (response.text)
```

O/P # Response

How can I help you with your public speaking today?

Step③: Streaming the Chatbot Response

Our examples so far generate responses in a non-streamed manner. This means that the endpoint would return a response object only after the model has generated the text in full. The longer the text is, the longer it takes to get back the response. If you are building an appⁿ, this directly impacts the user's perception of the appⁿ's latency.

↳ The Chat endpoint solves this probᵐ by supporting stream responses. In a streamed response, the endpoint would return a response object for each token as it is being generated. This means you can display the text incrementally w/o having to wait for the full completion.

To activate it, use, co.chat_stream() instead of co.chat()

In streaming mode, the endpoint will generate a series of objects. To get the actual text contents, we take objects whose event_type is text-generation.

If you have not already, make your own copy of the Google Colab notebook & run the code in this section to see the same example with streamed responses activated.

```
stream = co.chat_stream(message = "Hello. I'd like to
                        learn about techniques for effective
                        audience engagement",
                        model = "command-r-plus",
                        preamble = "You are an expert public coach"

for event in stream:
    if event.event-type == "text_generation":
        print(event.text, end = ' ')
```

Op

```
# Response (Streamed)

Sure! Here are some techniques for effective audience
engagement:

- ** Make eye contact: ** Look at - -
- ** Use gestures: **
```

Step ④ Building the Chat History

At the core of a conversation is a multi-turn dialog b/w the user & the chatbot. This requires the chatbot to have a "memory" of all the previous turns to maintain the state of the conversation.

Option ①: Using the Conversation History Persistence Feature

The chat endpoint supports state mgmt by persisting the conversation history. As a conversation progresses, the endpoint continuously updates the conversation history. This means developers don't have to deal with the complexity & inconvenience of managing conversation history in their app.

To use this feature, use the conversation-id parameter, which is a unique string you assign to a conversation. We'll use the uuid library to do this.

Putting everything together, let's now build a simple chat interface that takes in a user message, generates the chatbot response, automatically updates the conversation history, & repeats these steps until the user quits the conversation.

As described before, in streaming mode, the chat endpoint generates a series of objects. To get the conversation history, we take the object with event-type of "stream-end" & save it as a new variable chat_history

```
# Create a conversation ID
conversation_id = str(uuid.uuid4())
# Define the preamble
preamble = "You are an expert public speaking coach"
print ('Starting the chat. Type "quit" to end.\n')

while True:
    # User message
    message = input("User: ")
    # Typing "quit" ends the conversation
    if message.lower() == 'quit':
        print("Ending chat.")
        break
    # Chatbot response
    stream = co.chat_stream(message = message,
                            model= "command-r-plus",
                            preamble= preamble,
                            conversation_id= conversation_id)
    print("Chatbot: ", end = ' ')
    for event in stream:
        if event.event_type == "text-generation":
            print (event.text, end= '')
        if event.event_type == "stream-end":
            chat_history = event.response.chat_history
print(f"\n{'_' * 100}\n")
```

O/P

Resp

Sta

User

Chat

User

Ch

↳ Next,

for

O/P

Option ②:

If y

history per

parameter

O/P Response (Streamed)

Starting the chat. Type "quit" to end.

User: Hello

Chatbot: Hello! How can I help you today?

User:

Chatbot:

User:

Chatbot:

User: quit

Ending chat

↳ Next, we print the full conversation history.

```
for chat in chat_history:
    print(chat)
```

O/P role = 'USER' message = 'Hello' generation_id = '

role = 'CHATBOT' message = 'Hello! How can I help you today?'

Option ②: Managing the Conversation History Yourself

If you ~~not~~ opt not to use the endpoint's conversation history persistence feature, you can use the chat_history parameter to manage the conversation history yourself.

The chat history is a list of multiple turns of messages from the user & the chatbot. Each item is a cohere.ChatMessage object containing the role, which can be either "USER" or "CHATBOT" and the message containing the message string.

The following is an example of a chat history.

```
from cohere import ChatMessage
chat_history = [
    ChatMessage(role = "USER", message = "What is 2+2?"),
    ChatMessage(role = "CHATBOT", " " " The answer is 4")
    - - -
    - -
]
```

↳ The following modifies the prev. implementat^n by using chat-history instead of conversation-id for managing the conversat^n history.

```
# Initialize the chat history
chat_history = [ ]
# Define the preamble
preamble = "You are an expert public speaking coach"
print ('Starting the chat. Type "quit" to end. \n")
while True:
    # User message
    message = input ("User: ")
```

O/P

Re
Star
User
Chat

```
# Typing "quit" ends the conversation
if message.lower() == 'quit':
    print("Ending chat")
    break
# Chatbot response
stream = co.chat_stream(message = message,
                        model = "command-r-plus",
                        preamble = preamble,
                        chat_history = chat_history)

chat_response = ""
print("Chatbot: ", end = '')

for event in stream:
    if event.event_type == "text-generation":
        print(event.text, end = '')
        chatbot_response += event.text
print("\n").
# Add to chat history.
chat_history.extend(
        [ChatMessage(role = "USER",
                     message = message),
         ChatMessage(role = "CHATBOT",
                     message = chatbot_response)]

)
```

O/P

```
# Response (streamed)
Starting the chat. Type "quit" to end.
User : Hello.
ChatBot : _____
```

⇒ Parameters for Controlling Outputs

Here, you'll learn about the parameters that you can leverage to control the chat endpoint's outputs.

The chat endpoint is a versatile tool that empowers developers with an extensive array of options & parameters.

The Command model has many variations to select ~~them~~ from, where each has been carefully crafted to suit diff. needs.

Additionally, you will ~~learn~~ see how to use parameters to control the creativity of model responses.

Setup.

```
import cohere
co = cohere.Client("CohereAPI_key")
```

Model Type

With the chat endpoint, you can choose from several variations of the command model. Different models may produce different output profiles, so you may want to experiment with diff. models to get the best output for your use case.

Some of the models are as follows:

- **Command** : The default model used in a Chat endpoint call. If you don't define the **model** parameter, this model will be used.
- **Command-light**: A smaller, faster version of Command. Almost as capable, but a lot faster.